

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Институт информационных технологий

Кафедра микропроцессорных систем и сетей

А. А. Москалев, Н. И. Криштапович

АРХИТЕКТУРА ОПЕРАЦИОННЫХ СИСТЕМ

Лабораторный практикум
для слушателей переподготовки по специальности 1-40 01 73
«Программное обеспечение информационных систем»
всех форм обучения

В 4-х частях

Часть 2

***Автоматизация работы в консоли Windows
с помощью командных файлов***

Минск БГУИР 2011

УДК 004.451(076.5)
ББК 32.973.202я73
М82

Р е ц е н з е н т ы:

заведующий кафедрой менеджмента Минского института управления,
доктор технических наук, профессор В. А. Вишняков;
заведующий кафедрой программного обеспечения информационных технологий
Белорусского государственного университета информатики и радиоэлектроники,
кандидат технических наук, доцент В. В. Бахтизин

Москалев, А. А.
М82 Архитектура операционных систем : лаб. практ. для слушателей
переподготовки по спец. 1-40 01 73 «Программное обеспечение инфор-
мационных систем» всех форм обуч. В 4 ч. Ч. 2 : Автоматизация работы
в консоли Windows с помощью командных файлов / А. А. Москалев,
Н. И. Криштапович. – Минск : БГУИР, 2011. – 24 с.

ISBN 978-985-488-748-7 (ч. 2).

Рассмотрены вопросы создания и использования командных файлов для ав-
томатизации управления OS Windows различных версий в консольном режиме.
Пособие может быть полезно как слушателям, выполняющим лабораторный прак-
тикум, так и преподавателям, осуществляющим руководство выполнением лабо-
раторных работ по курсу «Архитектура операционных систем».

Часть 1 издана в БГУИР в 2011 г.

УДК 004.451(076.5)
ББК 32.973.202я73

ISBN 978-985-488-748-7 (ч. 2)
ISBN 978-985-488-719-7

© Москалев А. А., Криштапович Н. И., 2011
© УО «Белорусский государственный
университет информатики
и радиоэлектроники», 2011

Содержание

Лабораторная работа «Автоматизация работы в консоли Windows с помощью пакетных файлов»	4
1 Цель работы	4
2 Теоретические сведения	4
2.1 Понятие командного файла	4
2.2 Использование в пакетных файлах замещаемых параметров.....	4
2.3 Специальные команды для пакетных файлов	7
3 Выполнение работы	19
3.1 Подготовка к занятию	19
3.2 Выполнение упражнений по созданию пакетных файлов	19
4 Задания для самостоятельной работы	20
4.1 Общие указания для всех вариантов заданий.....	20
4.2 Варианты заданий.....	21
5 Вопросы для самопроверки.....	22
Литература	23

Лабораторная работа

«Автоматизация работы в консоли Windows с помощью пакетных файлов»

1 *Цель работы*

Изучить возможность автоматизации управления операционной системой с помощью командных файлов. Научиться создавать командные файлы и применять специфичные для них команды консольного режима на практике.

2 *Теоретические сведения*

2.1 *Понятие командного файла*

Командные файлы, также называемые программами пакетной обработки, сценариями или пакетными файлами, позволяют облегчить выполнение утомительных и часто используемых операций. По своей внутренней структуре пакетный файл представляет собой неформатированный текстовый файл, содержащий одну или несколько консольных команд и имеющий расширение **.bat** или **.cmd**. Когда имя такого файла вводится в командной строке, программа **Cmd.exe** выполняет в определенном порядке команды, содержащиеся в файле.

В пакетный файл можно включить любую команду. Однако некоторые команды, такие как **for**, **goto** и **if**, ориентированы на их применение в пакетных файлах. Они позволяют выполнять обработку условий в пакетных файлах, организовывать циклы и переходы. Например, **if** позволяет запускать команды в зависимости от выполнения либо невыполнения заданного условия. Другие команды позволяют управлять вводом и выводом, а также запускать другие пакетные файлы.

2.2 *Использование в пакетных файлах замещаемых параметров*

Параметры, вводимые в командной строке при запуске пакетного файла на выполнение, позволяют задавать различные исходные данные сценария пакетного файла. В самом пакетном файле значения параметров передаются через переменные с именами **%0**, **%1**, **%2**, ... , **%9**. Выполняя сценарий командного файла, процессор **CMD** отыскивает все вхождения этих переменных и заменяет их текстом соответствующих параметров. Параметр **%0** меняется на имя пакетного файла – слово номер 0 в командной строке. Все вхождения **%1** заменяются на первый параметр (слово номер 1 в командой строке). Вхождения **%2** заменяются текстом второго параметра (слово номер 2 в командной строке) и т. д. Запись **%*** в тексте командного файла замещается текстом всех аргументов, которые передаются пакетному файлу, за исключением имени самого файла (параметр **%0**). Запись **%~n** замещается всеми параметрами, начиная с *n*-го

($2 \leq n \leq 9$). Процесс замены параметров их значениями называют расширением параметров.

Для доступа к параметрам с порядковым номером больше чем девять используется команда **shift**. Эта команда уменьшает, начиная с указанной позиции, номера слов командной строки. Например, если переменной **%1** соответствовало первое слово (номер 1), то после применения инструкции **shift** это слово получит номер 0, второе слово получит номер 1 и т. д. Следовательно, теперь переменной **%9** уже будет соответствовать десятое слово в командной строке.

В качестве примера рассмотрим команду копирования содержимого из одной папки в другую. Предположим, что имя пакетного файла **mybatch.bat** и имена папок будут задаваться значениями параметров при запуске этого файла для выполнения. Тогда для копирования необходимо в пакетном файле записать инструкцию вида **xcopy %1*.* %2**.

Если ввести команду **mybatch.bat C:\folder1 D:\folder2**, то в этом случае в процессе исполнения пакетного файла текст **xcopy %1*.* %2** будет заменен на **xcopy C:\folder1*.* D:\folder2** и будет выполнено копирование содержимого из папки **C:\folder1** в папку **D:\folder2**.

Запуск того же файла командой **mybatch.bat C:\Temp D:\Work** приведет к формированию текста **xcopy C:\Temp*.* D:\Work**.

Ссылки на параметры в пакетном файле могут содержать *модификаторы*. Модификаторы заставляют **Cmd** использовать информацию о текущем диске и каталоге как часть имени либо как полное имя файла или каталога, и обеспечивают, таким образом, расширенную интерпретацию параметров.

Символ модификатора указывается после символов процента (%) и тильды (~) непосредственно перед номером параметра (**Pn**): **%~модификатор Pn**.

В таблице 1 описаны модификаторы, используемые для расширения ссылок на параметры командной строки.

Возможно объединение нескольких модификаторов в записи одной ссылки на параметр. Это приводит к комбинированию производимого эффекта:

%~dp1 – замещает **%1** буквой диска и именем каталога;

%~nx1 – замещает **%1** именем и расширением файла;

%~dp\$PATH:1 – просматривает каталоги в переменной среды **PATH** и замещает **%1** на букву диска и путь для первого найденного файла;

%~ftza1 – замещает **%1** информацией о файле в формате вывода команды **dir**.

Во всех приведенных примерах вместо **%1** можно использовать другие допустимые ссылки на параметры (**%2**, **%3** и т. д.).

Примечание. Модификатор **%*** не используется в комбинации с модификатором **%~**. Конструкция **%~** должна завершаться допустимым номером аргумента.

Таблица 1 – Модификаторы расширения параметров

Модификатор	Описание эффекта модификатора
%~1	Удаление обрамляющих кавычек (" ") при расширении %1
%~f1	Замена %1 полным путем доступа к файлу
%~d1	Замена %1 буквой диска
%~p1	Замена %1 именем каталога
%~n1	Замена %1 именем файла
%~x1	Замена %1 расширением имени файла
%~s1	Замена путем, содержащим только короткие имена
%~a1	Замена %1 атрибутами файла
%~t1	Замена %1 датой и временем модификации файла
%~z1	Замена %1 размером файла
%~\$PATH:1	Поиск в каталогах, перечисленных в переменной среды PATH , файла, указанного в первом параметре командной строки и замена %1 полным именем первого найденного файла. Если переменная среды не определена или поиск не обнаружил файлов, модификатор выдает пустую строку

В правом столбце таблицы 2 приведены примеры расширения параметров для случая, когда некоторый пакетный файл **mybatch.cmd** с командами из левого столбца запускается командой **MyBatch "aaa.txt" bbb.txt**.

Таблица 2 – Примеры использования модификаторов расширения параметров командной строки

Команда в пакетном файле	Команда после расширения параметров
Echo 1 %1	Echo 1 "aaa.txt"
Echo 2 %~1	Echo 2 aaa.txt
Echo 3 %~f1	Echo 3 C:\Documents and Settings \Alamos\ My Documents\aaa.txt
Echo 4 %~d1	Echo 4 C:
Echo 5 %~p1	Echo 5 \Documents and Settings\Alamos\ My Documents\
Echo 6 %~n1	Echo 6 aaa
Echo 7 %~x1	Echo 7 .txt
Echo 8 %~s1	Echo 8 C:\DOCUME~1\Alamos\MYDOCU~1\aaa.txt
Echo 9 %~a1	Echo 9 --a-----
Echo 10 %~t1	Echo 10 30.08.2010 22:50
Echo 11 %~z1	Echo 11 76
Echo 12 %~\$PATH:2	Echo 12 C:\WINDOWS\system32\bbb.txt

2.3 Специальные команды для пакетных файлов

2.3.1 Команда Call

Действие команды – вызов пакетного файла из другого с возвратом в первый после завершения.

Синтаксис команды:

```
call [[диск:][путь]имя_файла [параметры_вызова]]  
[:метка [аргументы]],
```

где **[диск:][путь]имя_файла** – имя и местоположение вызываемого пакетного файла;

параметры_вызова – параметры командной строки, передаваемые вызываемому пакетному файлу. Могут содержать ссылки на параметры родительского сценария в диапазоне от %0 до %9 и ссылки на значения переменных среды (например %Temp%);

:метка – указывает метку, на которую должно быть передано управление в вызываемом файле пакетной обработки. При использовании команды **call** с этим параметром создается новый контекст пакетного файла, а управление передается инструкции, следующей за указанной меткой. Когда после перехода на метку первый раз встречается конец пакетного файла, управление возвращается в вызывающий файл на инструкцию, следующую за инструкцией **call**;

аргументы – данные командной строки, которые передаются в новый экземпляр программы пакетной обработки, начинающийся с **:метки**. Могут включать параметры командной строки, ссылки на параметры родительского файла и ссылки на переменные среды.

Примечание. Не допускается использовать с командой **call** символ конвейера (|) и символы перенаправления ввода-вывода (<, >, >>).

Приведем примеры вложенного вызова пакетного файла:

1) команда **call MyChildBatch1.cmd** выполняет вложенный вызов файла **MyChildBatch1.cmd** с последующим возвратом в родительский пакетный файл. Никаких параметров в вызываемый файл не передаётся;

2) если дочерний пакетный файл **MyChildBatch2.cmd** принимает два параметра и требуется передать дочернему пакету два пакетных параметра родительского файла, то в родительском пакетном файле можно использовать следующую команду:

```
call MyChildBatch2.cmd %1 %2 .
```

2.3.2 Команда *Echo*

Действие команды – вывод на экран сообщения или установка режима последующего вывода на экран сообщений команд. Вызванная без параметров, команда **echo** выводит состояние действующего режима вывода.

Синтаксис команды:

echo [{on|off}] [сообщение],

где {on|off} – включение или отключения режима отображения на экране выполняемых команд (включение или отключение «эхо–вывода» команд). Символ @, помещенный перед любой командой, подавляет эхо–вывод этой команды. Если требуется отключить режим отображения команд и при этом не выводить на экран строку самой команды **echo**, можно ввести символ @ перед командой: @**echo off**;

сообщение – отображаемый на экране текст. Команда **echo** без параметров выводит на экран пустую строку.

Чтобы при использовании команды **echo** включить в текст сообщения специальные символы, например, символы канала (|) или перенаправления (<,>), введите символ (^) непосредственно перед символом канала или перенаправления (например, ^>, ^< или ^|). Чтобы вывести символ (^), введите два этих символа подряд (^ ^).

Следующий пример представляет собой фрагмент пакетного файла, выводящий сообщение из трех строк на экран. Перед сообщением и после него выводится пустая строка:

```
echo off
echo .
echo Этот пакетный файл
echo демонстрирует
echo вывод с использованием echo
echo .
```

2.3.3 Команда *For*

Действие команды – выполнение для каждого значения из заданного множества команды, указанной в **For**.

Синтаксис команды:

for [ключи] %%переменная IN (множество)
DO команда [параметры],

где %%переменная – подставляемый в цикле параметр (переменная цикла). Запись %%переменная используется только в пакетных файлах. При выполнении команды **For** из командной строки используется формат %переменная. Имена переменных записываются с учетом регистра и не должны быть

цифрой (чтобы не путать со ссылкой на параметры командной строки). Примерами имен переменных могут быть %A, %B1, %C и т. п.;

множество – имена одного или нескольких файлов, каталогов, диапазон значений или набор текстовых строк, подлежащих обработке командой, заданной параметром <команда>. В именах файлов допускаются символы шаблона. Набор значений множества обязательно заключается в круглые скобки;

команда – команда, которая будет выполнена для каждого элемента, входящего в (множество);

параметры – параметры и ключи, которые используются с указанной командой и, как правило, включают переменную цикла.

Например, чтобы вывести содержимое всех файлов в текущем каталоге, имеющих расширение .doc или .txt, можно использовать в пакетном файле следующую команду:

```
for %%f IN (*.doc *.txt) DO type %f.
```

При выполнении этой команды имя каждого файла с расширением .doc или .txt в текущем каталоге будет подставляться вместо переменной %%f, пока не будет выведено содержимое всех файлов.

В режиме расширенной обработки команд выполнение команды **For** может варьироваться в зависимости от используемых ключей. Далее рассматриваются некоторые из возможных вариантов применения ключей в команде **For**.

Вариант 1

```
for /D %%переменная IN (множество)  
DO команда [параметры]
```

Если **множество** содержит имена с подстановочными знаками * и/или ?, то **команда** выполняется для всех подходящих имен каталогов, а не файлов.

Вариант 2

```
for /R [[диск:]путь] %%переменная IN (множество)  
DO команда [параметры]
```

Выполняется **команда** для каталога <[диск:]путь> и для всех подкаталогов этого пути. Если параметр <[диск:]путь> не указан, то выполнение начинается с текущего каталога.

Если **множество** содержит только точку (.), то переменная цикла принимает значения пути для всех существующих каталогов.

Вариант 3

```
for /L %%переменная IN (начало, шаг, конец)  
DO команда [параметры]
```

Выполняется инструкция **For** для множества, представляющего последовательность целых чисел с заданным началом, концом и шагом приращения.

Вариант 4

В зависимости от содержимого множества значений параметра цикла можно выделить три подварианта команды **For**:

а) множество значений параметра команды **For** содержит набор имен файлов. Файлы множества открываются и считываются построчно. Каждая строка разбивается на группы символов в зависимости от опций. Затем для каждой непустой группы символов выполняется инструкция **<команда [параметры]>**, указанная в **For**:

```
for /F ["опции"] %переменная IN (множество имен файлов) DO команда [параметры];
```

б) множество содержит непосредственно заданную в кавычках строку символов **<"строка">**. Команда **For** разбивает указанную строку на группы символов в зависимости от указанных опций. Затем для каждой непустой группы символов выполняется инструкция **<команда [параметры]>**, указанная в **For**:

```
for /F ["опции"] %переменная IN ("строка")  
DO команда [параметры];
```

в) в качестве значения параметра цикла рассматривается стандартный вывод указанной в скобках и прямых одиночных кавычках команды **<'cmd'>**. Инструкция **For** рассматривает стандартный вывод как строку символов и выполняет для неё те же действия, что и в случае (б):

```
FOR /F ["опции"] %переменная IN ('cmd')  
DO команда [параметры] .
```

Ключ **/F** выделяет из каждой строки первое слово, очищенное от окружающих его пробелов. Пустые строки в файле пропускаются. При этом по умолчанию разделителем групп символов являются пробел и табуляция. Мы можем переопределить правила разбора строки, задавая необязательный параметр **<"опции">**, который представляет собой заключенную в кавычки последовательность из следующих ключевых слов:

eol=c – указывает, что в строке после символа **<c>** идет комментарий;

skip=n – задает количество строк **<n>** в начале файла, которые нужно пропустить;

delims=xxx – определяет набор разделителей групп вместо символов пробел и табуляция, заданных по умолчанию;

tokens=x,y,m-n – указывает номера групп, которые нужно передать инструкции-параметру **<команда>** для обработки, **m-n** задает диапазон, ***** в

конец списка указывает, что нужно обрабатывать все оставшиеся группы до конца строки;

usebackq – включает новые правила использования кавычек: строки, заключенные в обратные кавычки, выполняются как команды, строки, заключенные в прямые одиночные кавычки, являются строкой литералов команды, а строки, заключенные в двойные кавычки, используются для выделения имен файлов в списках имен файлов.

Пример 1:

```
for /F "eol=; tokens=2,3* delims=, " %i in
myfile.txt) do @echo %i %j %k .
```

Эта команда анализирует каждую строку файла **myfile.txt**, игнорируя все элементы строки после точки с запятой (**eol=;**). Группы разделяются запятыми и пробелами (**delims=,**), команде **@echo** передаются три группы: 2-я группа в переменной **%i**, 3-я группа в переменной **%j** и все группы после нее в переменной **%k**. Если имена файлов содержат пробелы, их следует заключать в кавычки (например **"Имя файла"**). В этом случае, чтобы кавычки не рассматривались как ограничители символьной строки для разбора, необходимо присутствие опции **usebackq**. Переменная **%i** объявлена явно в инструкции **For**, а **%j** и **%k** объявлены неявно с помощью **<tokens=2,3*>**. С помощью **tokens** можно указать до 26 элементов. При этом не допускается использование переменных со значением кода символа, превышающим значение кода буквы **z**. Так как имена переменных команды **For** являются глобальными, то одновременно не может быть активно более 52 переменных.

Пример 2:

```
for /F "delims==" %i IN ('set') DO @echo %i .
```

Эта команда выводит на экран имена всех переменных среды, определенных в настоящее время в системе, – стандартный вывод команды **set**.

В команде **For** используется режим расширенной подстановки ссылок на переменные.

2.3.4 Команда Goto

Действие команды – передача управления в пакетной программе на строку, содержащую указанную в команде метку. Когда метка найдена, выполнение продолжается со следующей за ней строки.

Синтаксис команды:

```
goto метка ,
```

где **метка** – строка, к которой выполняется переход, оформленная как метка в пакетной программе.

Метка может включать пробелы, но не может включать другие разделители, такие как точка с запятой или знак равенства.

В команде **goto** используются только первые восемь символов метки. Например, следующие метки эквивалентны и рассматриваются как **:Variant0**:

```
:Variant0  
:Variant01  
:Variant02
```

Метка, заданная в команде, должна соответствовать метке в тексте пакетной программы. Метка в пакетной программе должна находиться в отдельной строке и начинаться с двоеточия. Если в пакетном файле не содержится заданной метки, программа будет остановлена, а на экран будет выведено сообщение: **«Метка не найдена»**. Из этого правила есть исключение. Если расширения команды включены (а по умолчанию они включены), и в команде **goto** используется слово **EOF**, то управление будет передано в конец файла текущего пакетного сценария для выхода из него без метки **:EOF** в конце файла.

Команду **goto** можно использовать совместно с другими командами для выполнения операций с условиями.

2.3.5 Команда *If*

Действие команды – выполняет ветвление хода выполнения в зависимости от результата обработки условия.

Синтаксис команды:

```
if [not] <условие> команда1 [[else] команда2],
```

где **not** – обращает истинность условия («ложь» в «истину» и наоборот);

команда1 – команда, которая должна быть выполнена в случае истинности условия;

команда2 – команда, выполняемая в случае **else**, т. е. когда условие не является истинным.

Если условие, заданное в команде **if**, выполняется, будет выполнена команда, следующая за условием. Если условие не выполняется, команда, заданная в операторе **if**, пропускается, а управление переходит к команде оператора **else**, если она задана.

В качестве значения элемента **<условие>** могут использоваться следующие выражения:

errorlevel <число> – условие, которое является истинным, если предыдущая команда, обработанная интерпретатором команд **Cmd.exe**, завершилась с кодом, равным или большим заданного числа;

стр1==стр2 – условие, которое является истинным, если строки **<стр1>** и **<стр2>** совпадают. Строки задаются явно или могут быть пакетными переменными (например **%1**). Какие-либо ограничители, например, в виде кавычек, для явно задаваемых строк не предусмотрены;

exist <имя_файла> – условие, которое является истинным, если существует файл с именем <имя_файла>.

Особенностью использования оператора **else** является то, что он должен размещаться в той же строке, что и команда **if**. Далее приведено несколько «неправильных» фрагментов с объяснением ошибок. Так, фрагмент:

```
IF EXIST F1.txt del F1.txt ELSE echo Не найден файл
```

не будет работать, поскольку команда **del** должна заканчиваться переводом строки. В фрагменте

```
IF EXIST F1.txt del F1.txt  
ELSE echo Не найден файл
```

вышеописанная ошибка исправлена, однако появилась другая. Этот фрагмент не будет работать, поскольку команда **else** должна находиться в той же строке, что и команда **if**.

Использование скобок позволяет избежать указанных проблем. Правильным будет следующий вариант:

```
IF EXIST F1.txt (del F1.txt) ELSE echo Не найден файл
```

В качестве примера использования условия **errorlevel** рассмотрим пакетный сценарий, который копирует содержимое файла **f1.txt** в файл **f2.txt** на диске **D:**.

```
echo off  
copy d:\f1.txt d:\f2.txt  
if not errorlevel 1 goto end  
echo Ошибка при копировании.  
:end  
@echo конец пакетной программы.
```

Если копирование прошло успешно, команда **goto** передает управление на метку **:end** и сообщение об ошибке выведено не будет.

Команда **if** не может быть использована непосредственно для проверки существования каталога, но в каждом каталоге существует специальный файл – устройство **NUL**. Следовательно, существование каталога может быть заменено проверкой существования файла **NUL** в заданном каталоге, как показано ниже:

```
if exist c:\mydir\nul goto process
```

Если расширения командного процессора разрешены, дополнительно можно использовать следующие выражения для условия:

[/i] стр1 <опср> стр2 – условие, истинность которого зависит от используемого оператора сравнения <опср> и ключа **/i**. Значениями оператора могут быть: **EQU** – равно; **NEQ** – не равно; **LSS** – меньше; **LEQ** – меньше или равно; **GTR** – больше; **GEQ** – больше или равно. Присутствие ключа **/i** обуславливает то, что сравнение строк выполняется без учета регистра символов.

Если **стр1** и **стр2** состоят из цифр, строки преобразуются в числа и выполняется сравнение чисел;

defined <переменная> – условие является истинным, если <переменная> является именем определенной в блоке среды переменной;

cmdextver <число> – условие, которое является истинным, если номер версии расширения командного процессора **Cmd.exe** равен или больше указанного числа. Если расширения командного процессора запрещены, то условие считается невыполненным.

Использование выражения **defined** позволяет особым образом использовать ссылки **%errorlevel%**, **%cmdcmdline%** и **%cmdextversion%**.

Когда программа завершается, она возвращает код завершения, который с помощью параметра **errorlevel** можно использовать для формирования условий. Помимо этого можно использовать запись **%errorlevel%**. Строка **%errorlevel%** замещается строковым представлением текущего значения **errorlevel**, если только в блоке окружения нет переменной среды с именем **ERRORLEVEL**. В противном случае будет использовано ее значение. В качестве примера далее приведен фрагмент сценария, реализующий ветвление в зависимости от кода завершения:

```
goto answer%errorlevel%
:answer0
echo Код возврата программы равен 0
goto end
:answer1
echo Код возврата программы равен 1
goto end
...
:end
```

Можно использовать ссылку **%errorlevel%** вместе с операторами сравнения. Например, следующим образом:

```
if %errorlevel% LEQ 1 goto metka
```

В этих примерах необходимо с помощью выполняемого предварительно вызова **defined %errorlevel%** убедиться в отсутствии в блоке окружения переменной **errorlevel**.

Ссылка **%cmdcmdline%** замещается исходной командной строкой, переданной **Cmd.exe**, если не существует переменной среды с именем **cmdcmdline**.

Ссылка **%cmdextversion%** замещается строковым представлением текущего значения версии расширения **Cmd.exe**, если в момент обработки не существует переменной среды с именем **cmdextversion**.

2.3.6 Команда Pause

Действие команды – приостанавливает выполнение пакетной программы с выводом сообщения об ожидании нажатия пользователем любой клавиши.

Синтаксис команды: **pause**

При выполнении команды **pause** будет выведено сообщение «**Для продолжения нажмите любую клавишу...**». После нажатия любой клавиши выполнение пакетного файла продолжится.

Для остановки работы пакетной программы может быть в любой момент нажата комбинация **ctrl+c**. В этом случае на экран будет выведено сообщение «**Завершить выполнение пакетного файла [Y(да)/N(нет)] ?**».

Если в ответ была нажата клавиша **Y** (подтверждение), пакетная программа будет закончена и управление вернется операционной системе.

2.3.7 Команда Rem

Действие команды – определяет строку в пакетном файле или файле настройки как комментарий.

Синтаксис команды:

rem [текст],

где **текст** – задает строку символов, используемую в качестве комментария.

Команда **rem** не выводит комментарии на экран. Для вывода текста на экран в пакетных файлах или в файле **Config.nt** необходимо использовать команду **echo on**.

Следующий пример показывает применение команды **rem** и комментариев в пакетном файле.

```
@echo off
rem Эта пакетная программа проверяет новые диски.
rem Она называется CHECKNEW.BAT.
rem
echo Вставьте новый диск в устройство B.
pause
chkdsk b:
```

Предположим, что в файл **Config.nt** требуется ввести поясняющий комментарий перед командой **prompt**. Для этого в файл **Config.nt** следует добавить следующие строки:

```
rem Задание вывода текущего каталога в приглашении
prompt $p$g
```

2.3.8 Команда Setlocal

Действие команды – задает начало локальной области для определения переменных среды в пакетном файле.

Синтаксис команды:

```
setlocal {enableextension | disableextensions} {enabledelayedexpansion | disabledelayedexpansion},
```

где **enableextension** – параметр, который включает расширения командного процессора до появления соответствующей команды **endlocal**, вне зависимости от состояния расширений командного процессора перед командой **setlocal**;

disableextensions – параметр, который выключает расширения командного процессора до появления соответствующей команды **endlocal**, вне зависимости от состояния расширений командного процессора перед командой **setlocal**.

enabledelayedexpansion – включает расширения переменной среды с задержкой до появления соответствующей команды **endlocal**, вне зависимости от состояния расширений командного процессора перед командой **setlocal**.

disabledelayedexpansion – выключает расширения переменных среды с задержкой до появления соответствующей команды **endlocal**, вне зависимости от состояния расширений командного процессора перед командой **setlocal**.

Команда **setlocal** не выполняется при использовании вне пакетного файла. Изменения среды, выполненные после **setlocal**, являются локальными для пакетного файла. **Cmd.exe** восстанавливает первоначальные параметры при обнаружении команды **endlocal** или достижении конца пакетного файла.

Допускается использование нескольких команд **setlocal** и **endlocal** в пакетной программе (так называемых *вложенных* команд).

Команда **setlocal** устанавливает значение **ERRORLEVEL**. При её выполнении с аргументами **{enableextension | disableextensions}** или **{enabledelayedexpansion | disabledelayedexpansion}** переменная **ERRORLEVEL** принимает значение «ноль» (0) в случае успеха. В противном случае ее значение равно 1. Этим можно воспользоваться в пакетных программах для проверки доступности расширений командного процессора, например:

```
verify other 2>nul  
setlocal enableextensions  
if errorlevel 1 echo Включение расширений невозможно
```

Так как команда **cmd** не устанавливает значение переменной **ERRORLEVEL**, когда расширения командного процессора отключены, команда **verify** устанавливает переменную **ERRORLEVEL** в ненулевое значение при использовании этой команды с недопустимым аргументом. Если после выполнения команды **setlocal** с аргументом **enableextensions** значение пере-

менной **ERRORLEVEL** равно 1, то это означает, что расширения командного процессора недоступны.

Локальное изменение переменных среды в пакетном файле может быть использовано, например, следующим образом:

```
rem *****Начало комментария*****
rem Эта программа запускает приложение superapp,
rem записывает вывод в файл и загружает этот файл в
rem Блокнот.
rem *****Конец комментария *****
@echo off
setlocal
path=g:\programs\superapp;%path%
call superapp>c:\superapp.out
endlocal
start notepad c:\superapp.out
```

2.3.9 Команда Endlocal

Действие команды – отменяет использование локального окружения в пакетной программе, восстанавливает значения переменных среды, имевшиеся до выполнения соответствующей команды **setlocal**.

Синтаксис команды: **endlocal**

Параметры отсутствуют.

Эта команда неявно выполняется при завершении пакетного файла.

Пример локального изменения переменных среды в пакетном файле:

```
@echo off
rem Эта программа запускает приложение superapp,
rem записывает вывод в файл и
rem загружает этот файл в программу Блокнот.
setlocal
path=g:\programs\superapp;%path%
call superapp >c:\superapp.out
endlocal
start notepad c:\superapp.out
```

2.3.10 Команда Shift

Действие команды – изменяет нумерацию пакетных параметров.

Синтаксис команды: **shift**

Если расширения командного процессора разрешены, команда **shift** может использоваться с ключом **/n**, который указывает на изменение нумерации, начиная с **n**-го аргумента, где **n** – число от нуля до восьми. Например, команда **shift /2** приведет к следующим изменениям в нумерации параметров: слову в командной строке, которому соответствовал номер 3, станет соот-

ветствовать номер 2; слову, имевшему номер 4, станет соответствовать номер 3 и т. д.; номера слов 0 и 1 останутся неизменными.

Команда **shift** изменяет соответствие между словами командной строки (параметрами) и переменными %0, %1, ..., %9, являющимися ссылками на параметры внутри пакетного файла. Если до выполнения **shift** было соответствие: параметр 0 – переменная %0, параметр 1 – переменная %1, ..., параметр 9 – переменная %9, то после выполнения **shift** соответствие между параметрами и переменными станет следующим: параметр 1 – переменная %0, параметр 2 – переменная %1, ..., параметр 10 – переменная %9. То есть происходит как бы сдвиг параметров в командной строке относительно переменных %0, %1, ..., %9 в пакетном файле. После сдвига параметров командой **shift** начальное значение переменной %0 не может быть восстановлено. Команда **shift** не влияет на пакетный параметр %*.

Сдвиг параметров оказывается полезным при написании пакетных файлов, выполняющих одну и ту же операцию над произвольным числом параметров. В следующем пакетном файле **Mycopy.bat** показано, как использовать команду **shift** для копирования набора файлов в заданный каталог. Пакетные параметры представлены именем каталога и произвольным числом имен файлов.

```
@echo off
rem MYCOPY.BAT копирует в каталог
rem произвольное число файлов.
rem Используется следующий синтаксис:
rem mycopy dir файл1 файл2 ...
set todir=%1
:getfile
shift
if "%1"==" " goto end
copy %1 %todir%
goto getfile
:end
set todir=
echo Работа закончена
```

3 *Выполнение работы*

3.1 *Подготовка к занятию*

При подготовке к занятию необходимо изучить теоретический материал по основам создания и применения пакетных файлов:

- понятие пакетного файла;
- назначение и синтаксис команд, ориентированных на использование в пакетных файлах : IF, FOR, ECHO, PAUSE, REM, GOTO;
- использование параметров в пакетных файлах;
- использование переменных блока окружения в пакетных файлах;
- организация повторяющихся действий в пакетных файлах;
- ветвление в ходе выполнения пакета;
- вложенный вызов командного процессора;
- перенаправление ввода, вывода и вывода ошибок.

3.2 *Выполнение упражнений по созданию пакетных файлов*

3.2.1 *Простой пакетный файл*

С помощью редактора Edit создайте пакетный файл с именем P1.bat, содержащий команды для формирования дерева директорий и копирования файлов в соответствии с заданием предыдущей лабораторной работы*.

Выполните команду P1 и убедитесь, что она реализует поставленную задачу. Выполните команду P1 еще раз, рассмотрите и объясните изменения в информации, выводимой при повторном выполнении.

Откройте в Проводнике вашу рабочую папку и удалите папки, созданные с помощью P1.

Откройте в редакторе файл P1.bat и вставьте в начало файла команду **@ECHO OFF**. Сохраните файл и выполните команду P1. Проанализируйте отличия в выполнении пакетного файла в этот раз от предыдущего.

Удалите созданное дерево директорий. Если Проводник не показывает созданные папки, возможно, требуется выполнить команду **Обновить**.

3.2.2 *Пакетный файл с параметрами*

Создайте пакетный файл P2.bat, предусматривающий ввод параметров в командной строке запуска этого файла на выполнение.

Этот файл должен копировать под своим именем файл, указанный в качестве первого параметра в команде P2, в директорию, указанную в качестве второго параметра. Необходимо обеспечить проверку наличия необходимых параметров, проверку существования копируемого файла и существования директо-

* Эта лабораторная работа приведена в Ч.1 данного лаб. практикума: Москалев, А. А. Архитектура операционных систем. В 4 ч. Ч.1 : Интерфейс пользователя в консольном режиме. – Минск : БГУИР, 2011. – 24с.

рии назначения. Следует выводить необходимые сообщения и завершать работу, если продолжение невозможно.

Проверьте выполнение команды P2 с различными значениями параметров. Обеспечьте тестирование с полным перебором всех вариантов сочетаний параметров.

3.2.3 Пакетный файл с обработкой списков параметров

Создайте пакетный файл P3 для запуска программ. Командные строки запускаемых программ передаются через параметры строки запуска P3. Количество параметров запускаемых программ заранее не определено. Например, запуск по очереди программ **Edit**, **Dir** и **Mem**, перечисленных со своими параметрами в командной строке запуска P3, как показано далее:

```
d:\work\dl>P3.bat Edit f1.txt ; Dir d:\work ; Mem /d.
```

3.2.4 Пакетный файл с вызовом другого пакетного файла

Создайте пакетный файл P4, осуществляющий копирование нескольких файлов в заданные директории. Файлы и директории задаются парами параметров: файл1 – директория1, файл2 – директория2 и т. д. Для выполнения копирования используйте пакетный файл P2.

3.2.5 Комплексное упражнение

Создать пакетный файл PDIR для получения информации о директории, задаваемой параметром номер 1 в командной строке, записи полученной информации в файл (имя файла – параметр номер 2 в командной строке), отбора из сохраненного набора строк всех записей о файлах с расширением **.txt** и сохранения отобранных записей в файле с именем, задаваемым параметром номер 3 в командной строке. Отобранные записи сохранять в обратном алфавитному порядку имен файлов.

4 Задания для самостоятельной работы

4.1 Общие указания для всех вариантов заданий

При выполнении и оформлении заданий необходимо выполнять следующие требования:

- для всех вариантов обязательной является проверка корректности входных параметров, даже если это не указано в тексте задания;
- необходимо выполнить тестирование разработанного сценария с использованием множества возможных сочетаний параметров;
- в отчете должна быть представлена схема алгоритма, реализуемого сценарием пакетного файла;

– выполненное задание представляется для проверки преподавателю в виде пакетного файла и отчета о самостоятельной работе, выполненного на листах формата А4. Отчет должен содержать титульный лист, текст задания, сценарий пакетного файла с необходимыми комментариями, изображение алгоритма в виде схемы программы, результаты тестирования, выводы.

4.2 Варианты заданий

Вариант 1. Создать командный файл P11.bat, выводящий на экран содержимое директорий, перечисленных в командной строке при запуске P11 на выполнение. Если в командной строке перечислено более пяти директорий, для вывода использовать пять первых.

Вариант 2. Создать командный файл P21.bat, запускающий на выполнение программы, перечисленные в командной строке (в количестве от 1-й до 15-й). На консоль выводить перед запуском каждой программы ее имя и делать приостановку.

Вариант 3. Создать командный файл P3.bat, выводящий на экран построчно слова из командной строки запуска P3 на выполнение. Слова вывести в алфавитном порядке. Каждому слову должен предшествовать порядковый номер, который сформируется в пакетном файле.

Вариант 4. Создать командный файл P4.bat, выводящий на экран построчно порядковые номера в количестве, задаваемом параметром в командной строке запуска P4 на выполнение и не превышающем значения 20. Контролировать корректность параметра.

Вариант 5. Создать командный файл P5.bat. В качестве первого параметра при запуске этого пакетного файла предполагается ввод одного из слов: Редактор(1), Память(2), Директория(3). В случае 1 необходимо запустить программу Edit. В случае 2 следует запустить программу Mem с ключом, заданным во втором параметре, а в случае 3 – обеспечить запись содержимого директории, заданной вторым параметром, в файл dir_3.txt, местонахождение которого определяется третьим параметром.

Вариант 6. Создать командный файл P61, который будет копировать указанные файлы в заданный каталог. Каталог и перечень любого количества файлов задаются в качестве параметров.

Вариант 7. Создать командный файл P71, осуществляющий поочередный (с приостановкой) запуск программ, перечисленных в командной строке запуска этого файла на выполнение. Обеспечить возможность передачи параметров запускаемым программам.

Вариант 8. Создать командный файл P81, обеспечивающий вывод информации из блока окружения: тип и номер процессора, версия системы, имя

компьютера, имя пользователя. Текст вывести в файл, задаваемый в качестве параметра. Если файл не задан, то вывести текст на экран.

Вариант 9. Создать пакетный файл P91, обеспечивающий поиск в указанном первым параметром файле строк, содержащих заданное вторым параметром слово. Найденные строки записать в алфавитном порядке в файл, заданный третьим параметром. Если третий параметр не задан, вывести текст на экран.

Вариант 10. Создать пакетный файл P10, обеспечивающий вывод на экран содержимого всех файлов с расширениями (.txt), (.htm), (.bat) из директории, задаваемой параметром в командной строке.

5 Вопросы для самопроверки

- 1 Дайте определение понятия «командный файл». Есть ли различия между понятиями «командный файл» и «пакетный файл»?
- 2 Как в командном файле делаются ссылки на параметры, передаваемые в командной строке запуска файла на выполнение?
- 3 Как проверить отсутствие параметров в командной строке запуска пакетного файла?
- 4 Можно ли указать в командной строке запуска пакетного файла более пятнадцати параметров?
- 5 Как в пакетном файле проверить, был ли код завершения предыдущей команды большим 4?
- 6 Как в пакетном файле организовать вывод текста на консоль?
- 7 Назначение, структура и использование команды **For**.
- 8 Назначение, структура и использование команды **Shift**.
- 9 Как можно прервать выполнение командного файла.
- 10 Как записывается и для чего применяется в пакетном файле метка?
- 11 Как в пакетном файле вывести на экран значение переменной **ABC** из блока окружения программы?
- 12 Как раскрывается в пакетном файле выражение **%errorlevel%** ?
- 13 Как убедиться в том, что включен расширенный режим командного процессора?
- 14 Как с использованием команды **if** и выражения **exist** проверить существование директории, имя которой задается в качестве первого параметра пакетного файла?

Литература

1 Станек, Уильям Р. Командная строка Microsoft Windows : справочник администратора / Уильям Р. Станек ; пер. с англ. – М. : Издательско-торговый дом «Русская Редакция», 2004. – 480 с.

2 Дибров, А. Особенности реализации командного процессора cmd.exe операционных систем WinNT / А. Дибров. – [Электронный ресурс]. – 2009. – Режим доступа: <http://www.rsdn.ru/article/winshell/NTCommandProcessor.xml>. – Дата доступа : 20.05.2011.

3 Командный файл [Электронный ресурс]. – 2010. – Режим доступа: <http://dic.academic.ru/dic.nsf/ruwiki/976118>. – Дата доступа : 20.05.2011.

4 Bat файлы [Электронный ресурс]. – 2001. – Режим доступа: www.firststeps.ru/msoffice/win/bats/bats1.html. – Дата доступа : 20.05.2011.

5 10 советов по использованию сценариев [Электронный ресурс]. – 2006. – Режим доступа : <http://www.osp.ru/win2000/2006/08/3829295>. – Дата доступа : 20.05.2011.

6 Запуск программ и команд в заданное время [Электронный ресурс]. – 2006. – Режим доступа: <http://www.securitylab.ru/processinfo/267532.php>. – Дата доступа : 20.05.2011.

7 Фигурнов, В. Э. IBM PC для пользователя / В. Э. Фигурнов. – 2-е изд. – М. : Финансы и статистика, 1991 (и более поздние издания).

Учебное издание

Москалев Александр Анатольевич
Криштапович Наталия Иосифовна

Архитектура операционных систем

Лабораторный практикум
для слушателей переподготовки по специальности 1-40 01 73
«Программное обеспечение информационных систем»
всех форм обучения

В 4-х частях

Часть 2

Автоматизация работы в консоли Windows с помощью командных файлов

Редактор Г. С. Корбут
Корректор А. В. Тюхай

Подписано в печать 13.7.2011.	Формат 60х84 1/16.	Бумага офсетная.
Гарнитура «Таймс».	Отпечатано на ризографе.	Усл. печ. л.
Уч.-изд. л. 1,4.	Тираж 100 экз.	Заказ 418.

Издатель и полиграфическое исполнение: учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»
ЛИ №02330/0494371 от 16.03.2009. ЛП №02330/0494175 от 03.04.2009.
220013, Минск, П. Бровки, 6